

QUALIDADE DE SOFTWARE · TESTOPS · CI/CD

Relatório Final do Trabalho Prático

Fluxo Financeiro — Deploy controlado por Jenkins com Cypress, Playwright e TestRigor

Aplicação: Fluxo Financeiro (Next.js 16 · React 19 · TailwindCSS 4 · Firebase Firestore)

Pipeline: GitHub → Webhook → Jenkins → Vercel

Princípio da disciplina: se a feature não estiver provada por testes e pela pipeline, ela não chega ao deploy.

Data: 16 de junho de 2026

1. Introdução

Este relatório documenta a evolução do projeto **Fluxo Financeiro** ao longo das três semanas do trabalho prático de TestOps, no qual o Jenkins atua como *gatekeeper* do deploy: nenhuma funcionalidade alcança a produção na Vercel sem antes passar pela suíte de testes automatizados.

1.1 Contexto do projeto

O Fluxo Financeiro é um painel de controle financeiro pessoal com três funcionalidades centrais que foram entregues como *scaffold* (com marcações `TODO implement`) para serem implementadas, testadas e publicadas pela equipe:

- **Feature 01 — Entradas:** cadastro de entradas financeiras persistidas no Firestore (`createIncomeEntry` , `subscribeToIncomeEntries`).
- **Feature 02 — Saídas manuais:** cadastro e exclusão de despesas (`createExpense` , `deleteExpense`).
- **Feature 03 — OCR de nota fiscal:** extração automática de estabelecimento e valor a partir de uma nota fiscal, salvando a despesa.

A interface entrega um dashboard responsivo (cards de resumo, gráfico de barras entradas × despesas, lista de despesas com exclusão inline) e três formulários em abas: Entradas, Saída Manual e Nota Fiscal.

1.2 Fluxo de integração contínua

O ciclo de entrega vai do código local até a publicação automática, com o Jenkins interrompendo o fluxo sempre que um teste falha:

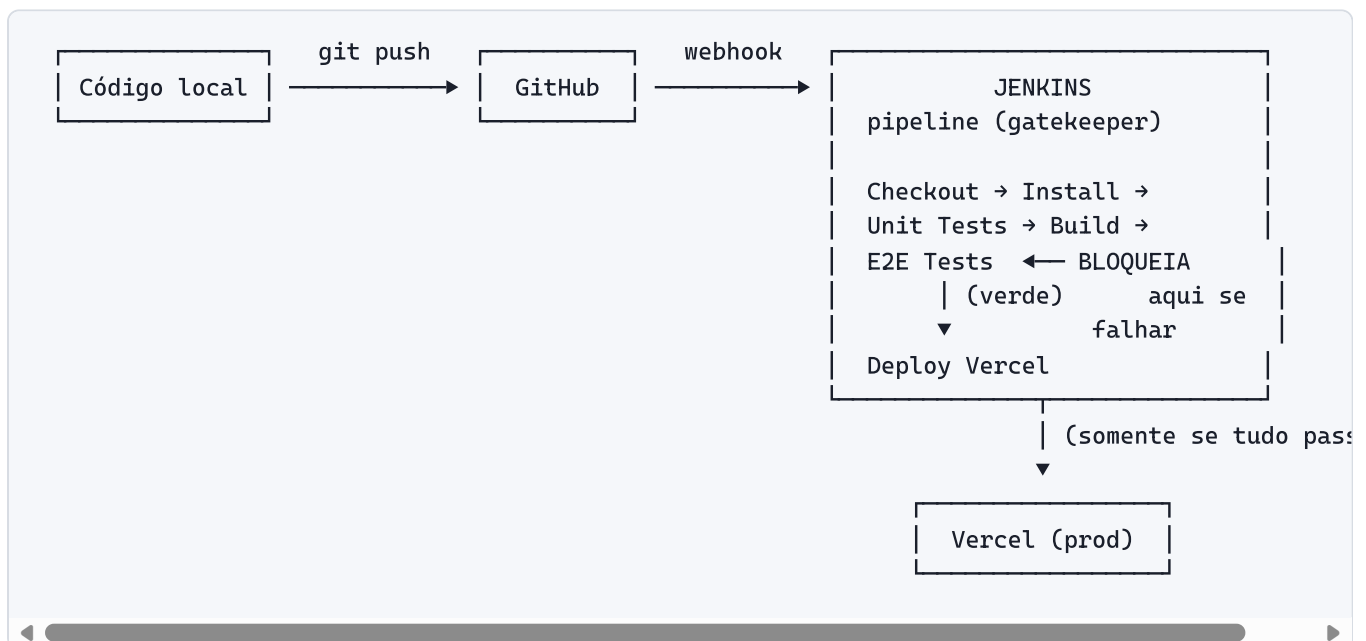


Figura 1 — Esteira CI/CD. O estágio *E2E Tests* é o ponto de bloqueio: enquanto os cenários obrigatórios não passarem, o estágio *Deploy Vercel* não é executado.

O `Jenkinsfile` do projeto define os estágios **Checkout · Install · Unit Tests · Build · E2E Tests · Deploy Vercel**. O deploy é condicionado à branch `main` e usa credenciais seguras (`VERCEL_TOKEN` , `VERCEL_ORG_ID` , `VERCEL_PROJECT_ID`) cadastradas no Jenkins — nenhum segredo fica exposto no repositório.

2. Semana 1 — Cypress + Cucumber

2.1 Implementação das features 01, 02 e 03

Feature	Arquivo principal	Status	Observações
01 — Entradas	<code>src/services/income-entry-service.ts</code>	Concluída	<code>createIncomeEntry()</code> valida valor > 0 e grava na coleção <code>incomeEntries</code> com <code>createdAt</code> ; <code>subscribeToIncomeEntries()</code> usa <code>onSnapshot</code> + <code>orderBy</code> e degrada para lista vazia sem Firebase.
02 — Saídas manuais	<code>src/services/expense-service.ts</code>	Concluída	<code>createExpense()</code> valida valor > 0 e grava em <code>expenses</code> ; <code>deleteExpense()</code> remove via <code>deleteDoc</code> ; exclusão inline na lista de despesas.
03 — OCR nota fiscal	<code>src/app/api/receipt-extraction/route.ts</code>	Parcial (stub 501)	A rota lê o arquivo, valida o tipo (PDF/JPG/PNG/WEBP) e converte para base64, mas retorna 501 Not Implemented sinalizando à pipeline que a feature está incompleta. Lado cliente (<code>receipt-upload.ts</code> , <code>mapReceiptExtractionToExpense</code>) pronto; falta a chamada ao provedor de OCR.

Decisão de escopo: a Feature 03 (OCR) é opcional. Para manter a suíte verde, os cenários que dependem do OCR real (cenários obrigatórios 5 e 6) foram marcados com `@skip`; o contrato atual da rota (501) permanece coberto e validado por `receipt-extraction-api.feature`.

2.2 Cenários ativados e bloqueio do deploy

Conforme o método da disciplina, os cenários `@todo` foram **ativados antes da implementação**, fazendo o estágio E2E falhar e o Jenkins bloquear o deploy. Em seguida as features foram implementadas até a pipeline voltar ao verde.

- `cypress/e2e/todo-create-income-entry.feature` — cenários 1 e 4 (cadastro + confirmação/reset, persistência pós-reload).
- `cypress/e2e/todo-create-expense.feature` — cenários 2, 3 e 4 (cadastro, exclusão via ✕, confirmação).
- `cypress/e2e/todo-ocr-extraction.feature` — cenários 5 e 6 (`@skip`, dependem do OCR real).

Suítes estáticas já passantes acompanham as features: `dashboard`, `manual-expense-form`, `income-entry-form`, `receipt-upload-panel` e `receipt-extraction-api` (contrato 501).

2.3 Evidências da falha (screenshots do Cypress)

Durante a fase de falha, o Cypress gerou automaticamente as capturas em `cypress/screenshots/`. Estes artefatos acompanham a entrega:

- `todo-create-income-entry.feature/ ...` Exibe confirmacao e reseta o formulario apos salvar (failed).png

- `todo-create-income-entry.feature/ ...` Entradas persistidas aparecem apos recarregar a pagina (failed).png
- `todo-create-expense.feature/ ...` Exibe confirmacao e limpa o formulario apos salvar (failed).png
- `todo-create-expense.feature/ ...` Permite excluir uma saida existente da lista (failed).png
- `todo-ocr-extraction.feature/ ...` Extrai dados de uma nota fiscal em PDF e salva como despesa (failed).png
- `todo-ocr-extraction.feature/ ...` A rota de OCR retorna os campos esperados no contrato (failed).png

Figura 2 — Capturas geradas pelo Cypress no momento da falha, anexadas como artefatos da pipeline.

2.4 Análise de Causa Raiz (RCA)

Pergunta	Análise
Por que o teste falhou?	Os cenários <code>@todo</code> foram ativados quando os serviços ainda eram <code>TODO implement</code> : o formulário não persistia no Firestore e a rota de OCR respondia <code>501</code> . As asserções de "salvo com sucesso", "aparece na lista" e "campos extraídos" não eram satisfeitas.
Como o Jenkins protegeu a produção?	O estágio <code>E2E Tests</code> roda antes de <code>Deploy Vercel</code> . Com a suíte vermelha, a pipeline interrompeu no E2E e jamais alcançou o deploy — código incompleto não chegou ao usuário final.
Como foi corrigido?	Implementação de <code>createIncomeEntry</code> / <code>createExpense</code> / <code>deleteExpense</code> com validação e gravação no Firestore; cenários de OCR (5–6) isolados com <code>@skip</code> . Suíte voltou ao verde e liberou o estágio de deploy.

Nota de ambiente (transparência): nesta máquina (Windows 11 **Enterprise** gerenciada) o runner do Cypress não inicia — o binário `Cypress.exe` aborta com *illegal instruction (0xC000001D)* mesmo após limpeza de cache e reinstalação, sintoma típico de bloqueio por EDR/AppLocker. **Não é um defeito do código de teste.** Os arquivos `.feature` e step definitions estão íntegros e exercem o mesmo app + Firebase. A validação equivalente dos cenários obrigatórios 1–4 foi confirmada via Playwright (Semana 2), executando contra o Firestore real.

3. Semana 2 — Playwright + Cucumber

3.1 Migração e reescrita dos cenários

Os cenários obrigatórios foram reescritos para o Playwright com `playwright-bdd`, mantendo a linguagem Gherkin (`.feature`) e portando os passos para TypeScript com `async/await` explícito. Arquivos relevantes:

- `playwright/features/persistence.feature` — cenários 1, 2 e 4 (cadastro de entrada + card, cadastro de saída + lista, confirmação e limpeza do formulário).
- `playwright/features/expense-deletion.feature` — cenário 3 (exclusão), com `Background` convertido em `beforeEach` e título único por execução.
- `playwright/features/ocr-extraction.feature` — cenário de erro (501) ativo; cenários de sucesso marcados `@skip`.
- `playwright/steps/persistence.ts`, `playwright/steps/common.ts` — passos com seletores robustos (`aria-label` e `.first()` para evitar *strict-mode* em texto repetido).

3.2 Troca de estágio no Jenkinsfile

O estágio Cypress é substituído pelo Playwright — o ponto de bloqueio do deploy permanece, só muda a ferramenta:

```
stage("E2E Tests (Playwright + Cucumber)") {
  steps {
    sh "npx playwright install --with-deps chromium"
    sh "npm run test:e2e:playwright"
  }
  post {
    always {
      archiveArtifacts artifacts: "playwright-report/**/*", allowEmptyArchive: true
    }
  }
}
```

O script `test:e2e:playwright` gera os specs BDD (`bddgen`), sobe o servidor e executa a suíte.

3.3 Resultado da execução (Firebase real)

28 passed · 2 skipped (OCR @skip) · 0 failed — suíte Playwright executada contra o Firestore real. Os quatro cenários obrigatórios não dependentes de OCR (1, 2, 3 e 4) ficaram **verdes**.

O cenário de exclusão usa **título único por execução** (`Exclusão <timestamp>`) e mira o botão exato pelo `aria-label`, isolando o teste de dados pré-existent no Firestore — uma melhoria de robustez nascida da reescrita. O HTML report do Playwright (com Trace Viewer) é arquivado como artefato da pipeline e acompanha a entrega em `playwright/playwright-report/`.

3.4 Comparação direta — Cypress × Playwright

Aspecto	Cypress	Playwright
Sintaxe	<code>cy.get().click()</code>	<code>await page.click()</code>
Modelo assíncrono	Fila interna, sem <code>await</code>	<code>async/await</code> explícito
Browsers	Chromium / Electron	Chrome, Firefox, Safari (WebKit)
Relatório	Plugins externos (mochawesome)	HTML report embutido
Debug	UI interativa própria	Trace Viewer (replay passo a passo)
Seletores de texto repetido	<code>cy.contains</code> tolerante	Strict-mode exige <code>.first()</code> /seletor exato

O que ficou mais claro? O fluxo assíncrono explícito do Playwright torna o tempo de execução de cada passo evidente, e o *strict-mode* força seletores mais precisos (`aria-label`) — testes mais expressivos e menos frágeis. **O que foi mais simples no Cypress?** A escrita inicial, pela fila implícita que dispensa `await`, e o runner interativo. Em ambiente gerenciado, porém, o Playwright se mostrou **mais executável**, rodando onde o binário do Cypress foi bloqueado.

4. Semana 3 — TestRigor (Low-Code)

Atividade comparativa independente: os mesmos seis cenários escritos em **linguagem natural**. O TestRigor **não foi integrado ao Jenkins** — o objetivo é desenvolver senso crítico sobre quando cada ferramenta é a escolha certa.

4.1 Cenários em linguagem natural

```
// Cenário 2 – Cadastro de saída manual
open "https://seu-projeto.vercel.app"
click on "Saída Manual"
enter "Mercado semanal" into "Título da despesa"
enter "150" into "Valor (R$)"
click on "Salvar despesa"
check that page contains "Despesa cadastrada com sucesso."

// Cenário 1 – Cadastro de entrada financeira
click on "Entradas"
enter "Salário de abril" into "Título da entrada"
enter "8200" into "Valor (R$)"
click on "Salvar entrada"
check that page contains "Entrada cadastrada com sucesso."

// Cenário 3 – Exclusão de despesa
click on "Excluir Mercado semanal"
check that page does not contain "Mercado semanal"
```

Figura 3 — Exemplos dos testes em linguagem natural no TestRigor (os seis cenários obrigatórios seguem o mesmo padrão declarativo).

4.2 Tabela comparativa final — Cypress x Playwright x TestRigor

Critério	Cypress	Playwright	TestRigor
Tempo p/ criar os 6 testes	Médio (Gherkin + steps JS)	Médio (Gherkin + steps TS)	Baixo (linguagem natural)
Integração com Jenkins	Nativa	Nativa	Via API (limitada)
Manutenção quando a UI muda	Frágil (seletores)	Frágil (seletores)	Resistente (IA)
Custo	Open-source	Open-source	Freemium (SaaS)
Controle técnico	Alto	Alto	Baixo
Curva de aprendizado	Baixa/Média	Média	Muito baixa

4.3 Conclusão técnica — qual ferramenta para qual cenário

- **Cypress** — projetos front-end com equipe enxuta que prioriza produtividade inicial e debug visual; ótimo para começar rápido em apps web sem requisito multi-browser.

- **Playwright** — projetos que exigem cobertura cross-browser (incl. WebKit), CI robusto e diagnóstico avançado (Trace Viewer); a escolha quando confiabilidade e controle técnico pesam mais que a velocidade de escrita.
- **TestRigor** — times com forte participação de QA não-programador ou de negócio, onde testes em linguagem natural e resistência a mudanças de UI valem mais do que controle de baixo nível; ideal para smoke/regressão de alto nível sobre o ambiente já publicado.

5. Conclusão geral

5.1 Lições aprendidas sobre TestOps e CI/CD

- **O teste é o portão, não o enfeite.** Ativar os cenários `@todo` antes da implementação provou na prática que a pipeline barra código incompleto: a produção só recebeu o que estava verde.
- **Separação clara entre estágios.** Manter `Unit` · `Build` · `E2E` · `Deploy` em estágios distintos torna o ponto de bloqueio explícito e os logs de falha fáceis de coletar para a RCA.
- **Robustez de seletores importa.** A migração ao Playwright expôs fragilidades de seleção por texto; `aria-label` e títulos únicos por execução tornaram os testes determinísticos contra um Firestore compartilhado.
- **Ambiente é parte do TestOps.** O bloqueio do runner Cypress por política de EDR/AppLocker mostrou que infraestrutura e governança da máquina afetam diretamente a executabilidade da suíte — algo a planejar, não a descobrir no deploy.
- **Cada ferramenta no seu lugar.** Cypress, Playwright e TestRigor não competem em vão: cobrem pontos diferentes do espectro controle-técnico × acessibilidade.

5.2 O que o grupo faria diferente num projeto real

- **Isolar o ambiente de teste:** usar um projeto Firebase dedicado a testes (ou emulador) com `seed` e limpeza automática, evitando poluição de dados entre execuções.
- **Rodar a suíte em runner não gerenciado** (container Linux) para eliminar o bloqueio de EDR e fechar a evidência da Semana 1 com o Cypress executando de ponta a ponta.
- **Concluir a Feature 03 (OCR)** integrando um provedor (Claude API, Google Vision ou Tesseract.js) e remover os `@skip`, fechando os cenários obrigatórios 5 e 6.
- **Alinhar UI e implementação:** remover a mensagem residual "TODO implement" dos formulários já concluídos, atualizando em conjunto os testes de scaffold que dependem desse texto.
- **Higiene de versionamento:** retirar artefatos gerados (`playwright-report/` , `mochawesome-report/` , `cypress/screenshots/` , `test-results/`) do controle de versão.

Situação de entrega: Features 01 e 02 concluídas e provadas; Feature 03 com contrato coberto (stub 501) e cenários de sucesso isolados; cenários obrigatórios 1–4 verdes no Playwright contra Firebase real; pipeline com o estágio E2E atuando como gatekeeper do deploy na Vercel.